SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE ULHASNAGAR – 421 003

CERTIFICATE

This is to certify that Mr./Ms. Abhishek Umashankar Bhardwaj of S.Y. Information Technology (SYIT) Roll No. 2542004 has satisfactorily completed the Open Source DataBase Management System Mini Project entitled Library Book Issue & Fine Management System during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	3
2	Problem Definition	4
3	Objectives of the Project	5
4	Scope of the Project	6
5	Requirement Specification	7
6	System Design	8
7	Database Design	9
8	UML Diagrams	12
9	SQL Implementation	16
10	System Testing and Result	20
11	Security, Backup and Recovery	22
12	Future Scope and Conclusion	23
13	References	25
14	Glossary	26

CHAPTER 1: INTRODUCTION

In the context of the **Library Book Issue & Fine Management System**, the database system stores information related to books, library members, book issuing records, and fines. Each table in the database represents a real-world entity. For example, the *Books* table stores information about available books, the *Members* table stores user details, the *Issue* table tracks book transactions, and the *Fine* table stores penalty records for late returns.

A database system also supports operations such as inserting new records, updating existing records, deleting unnecessary records, and retrieving specific information using SQL queries. It ensures data integrity using constraints like primary keys and foreign keys. By using a database system, manual errors are minimized, data redundancy is reduced, and record maintenance becomes easier. Overall, database systems form the backbone of modern data-driven applications and are widely used in banking, education, healthcare, and business environments.

The Library Book Issue & Fine Management System addresses the problem of inefficient manual record handling in libraries. Traditionally, many small libraries maintain book records, issue registers, and fine calculations manually using notebooks or spreadsheets. This method is time-consuming, error-prone, and difficult to maintain as the number of books and members increases. Managing book availability manually can lead to confusion, especially when multiple books are issued and returned daily.

The proposed database system solves these problems by providing a centralized digital solution where all data is stored in structured tables. It helps track book availability in real time by updating available copies whenever a book is issued or returned. It also automates fine calculation based on late return days, reducing manual calculation errors. The system ensures that each member and book has a unique identification number, which helps in quick searching and tracking of records.

Another major problem addressed is data inconsistency. In manual systems, the same information may be written multiple times, leading to duplication and mismatched records. The database eliminates this issue by storing data in related tables connected through foreign keys. It also improves data security by restricting unauthorized access. The system is scalable, meaning it can handle increasing numbers of books and members without performance issues. Therefore, this system provides a reliable, efficient, and automated solution for library management.

CHAPTER 2: PROBLEM DEFINITION

In many small and medium-sized libraries, book record management, member tracking, book issuing, return monitoring, and fine calculation are still handled manually or through unstructured digital methods such as spreadsheets. While these methods may work for a small amount of data, they become inefficient and difficult to manage as the number of books and members increases. Manual systems require staff to record every transaction in registers, which is time-consuming and prone to human errors. Searching for specific records, such as which member has issued a particular book or which books are overdue, becomes difficult and slow. This creates operational delays and reduces the overall efficiency of the library system.

In an existing manual or semi-manual system, data is often written repeatedly in multiple places. For example, book issue records may be maintained separately from member records and fine records. This leads to redundancy and increases the chances of mismatched or inconsistent data. If a mistake occurs while updating one record, it may not be corrected in other related records. Additionally, manual fine calculation for late returns can result in incorrect fine amounts, which can cause financial loss or disputes with members. There is also very limited security in manual systems because physical registers can be accessed by anyone, leading to risks of data loss or unauthorized modifications.

The major issues in traditional systems include:

•Data redundancy due to repeated manual entries
• Data inconsistency between different registers or files
• Difficulty in tracking real-time book availability
• Manual fine calculation errors
• Lack of proper data security and access control
• Difficulty in generating reports and summaries
• Time-consuming manual searching and updating of records

Due to these challenges, there is a strong need for a database-driven solution. A structured database system ensures that all data is stored in an organized format using tables and relationships. In the Library Book Issue & Fine Management System, each book and member is assigned a unique ID, which helps in fast searching and accurate record tracking. The system automatically updates book availability when a book is issued or returned. It also calculates fines automatically based on due date and return date, which reduces manual workload and errors.

A database-driven system also improves data security by allowing only authorized users to access or modify data. SQL queries make it possible to generate reports quickly, such as issued book lists or pending fine details. Therefore, implementing a relational database system using MySQL provides an efficient, reliable, and secure solution for modern library management needs.

CHAPTER 3: OBJECTIVES OF THE PROJECT

The main objective of the **Library Book Issue & Fine Management System** project is to design and develop a structured database solution that can efficiently manage library operations such as book storage, member record management, book issuing, return tracking, and fine calculation. Traditional library systems often rely on manual record keeping, which is time-consuming and prone to errors. This project aims to replace such systems with a reliable digital database solution using MySQL. By implementing this system, library data can be stored in an organized format, making it easier to retrieve, update, and maintain records. The system is designed to ensure data accuracy, reduce redundancy, and provide faster processing of library transactions.

One of the primary objectives of this project is to design a structured database system. The project involves creating relational tables such as Books, Members, Issue, and Fine. Each table represents a real-world entity and stores specific types of data. The relational structure helps maintain proper relationships between data using foreign keys. This structured design ensures that the database remains organized and easy to manage even when data volume increases.

Another important objective is to implement SQL queries for data management. SQL queries such as INSERT, UPDATE, DELETE, and SELECT are used to perform database operations. JOIN queries help in retrieving combined data from multiple tables, such as showing which member issued which book. These queries make data management efficient and help generate reports quickly.

The project also focuses on ensuring data integrity using constraints. Primary keys are used to uniquely identify each record, while foreign keys maintain relationships between tables. NOT NULL constraints ensure that important fields cannot be left empty. These constraints help maintain database accuracy and reliability.

Another objective is to gain hands-on experience with MySQL. Through this project, users learn how to create databases, design tables, apply constraints, insert data, and retrieve information using queries. This practical exposure helps in understanding real-world database applications and improves technical skills required in IT and software development fields.

CHAPTER 4: SCOPE OF THE PROJECT

The **Library Book Issue & Fine Management System** is designed to provide an efficient and structured solution for managing library operations using a relational database. The system is mainly focused on storing and managing data related to books, library members, book issue transactions, return tracking, and fine calculation. This project can be implemented in environments where maintaining book records and tracking borrowing activities are essential. The system ensures real-time data updates, reduces manual workload, and improves overall operational efficiency. By using MySQL, the system provides a reliable and scalable platform that can handle increasing amounts of data without affecting performance. The scope of this project mainly lies in educational institutions, small organizations, and places where book inventory management is required.

The system can be used in multiple areas where record management is necessary. It is especially useful in school libraries, college libraries, and training institute libraries. It can also be adapted for book rental shops and small businesses that lend books or study materials. Since the system is database-driven, it can store large volumes of data and allows fast searching of records. The system can generate useful information such as issued books list, overdue books list, and pending fine details. This makes the system useful not only for record storage but also for reporting and monitoring purposes.

The main users of this system include:

- **Admin** – Responsible for database management, data monitoring, and user control
- **Library Staff** – Handles book issue, return entry, and fine updates
- **Students / Members** – Borrow books and return them within due date
- **Business Users** – Can adapt the system for inventory tracking and customer record management

The system scope can also be divided into subdomains:

- **Educational Use** – Managing student book records in schools and colleges
- **Business Use** – Managing book rental or material lending businesses
- **Administrative Use** – Maintaining structured records for organizational use

However, this project has some academic limitations. It is mainly focused on database-level implementation and does not include web interface or mobile application integration. It also does not include advanced analytics or automated notification systems. Despite these limitations, the system provides a strong foundation for realworld database application development.

CHAPTER 5: REQUIREMENT SPECIFICATION

Hardware Requirements:

• Computer/Laptop

• Minimum 4GB RAM

• 20GB free storage

Software Requirements:

• MySQL Server

• MySQL Workbench

• Windows/Linux OS

• SQL language

CHAPTER 6: SYSTEM DESIGN

System Design describes how the Library Book Issue & Fine Management System is organized and how different components work together to perform database operations. The system is designed using a database centered approach where MySQL acts as the core component for storing and managing data. The main purpose of system design is to ensure that data is stored properly, processed efficiently, and retrieved quickly when required. In this project, the system is designed to manage book records, member records, book issue transactions, return updates, and fine calculation. The design ensures that manual work is reduced and database operations are performed automatically using SQL queries.

The system allows users such as admin and library staff to interact with the database using MySQL Workbench. The design ensures secure and controlled access to data. Each operation such as book issue or return is performed using SQL queries. The system is designed to maintain data integrity using constraints such as primary keys and foreign keys. It also ensures that data is updated in real time. The system design focuses only on database interaction and query execution process, not hardware components. This design makes the system efficient, reliable, and easy to maintain.

• Users such as Admin and Library Staff interact with the database using SQL queries

• Queries are written and executed using MySQL Workbench

• Users can perform operations like inserting new records, updating data, deleting records, and retrieving information

• The system allows multiple users to access the database at the same time

• MySQL Server stores all database tables and records Stores all database tables such as Books, Members, Issue, and Fine

• Processes SQL queries received from users

• Maintains data relationships using foreign key constraints

• Ensures data accuracy using primary key and NOT NULL constraints

• Handles multiple requests from different users simultaneously

• Provides security and access control to prevent unauthorized data access

• Optimizes query execution for better performance• User writes query in MySQL Workbench

• User writes query in MySQL Workbench

• Query is sent to MySQL Server

• MySQL Server checks query syntax and constraints

• Required operation is performed on database tables

• Result is generated and sent back to user

CHAPTER 7: DATABASE DESIGN

Database Design is one of the most important parts of the Library Book Issue & Fine Management System. It defines how data is stored, organized, and related inside the database. A good database design ensures data consistency, reduces redundancy, and improves query performance. In this project, the database is designed using a relational model where data is stored in tables and relationships are maintained using keys. The database contains four main tables: Books, Members, Issue, and Fine. Each table represents a real-world entity and stores specific types of data. The tables are connected using foreign keys to maintain relationships between data.

The database is designed based on real library operations. The Books table stores book details and availability status. The Members table stores member details. The Issue table stores book transaction details like issue date and return date. The Fine table stores late fine details. The database design ensures that each record is unique and properly linked. Constraints such as Primary Key, Foreign Key, and NOT NULL are used to maintain data integrity. This design allows easy data retrieval using SQL queries and supports future database expansion.

7.1 Entity Description

Books Table

The Books table stores information about books available in the library. It includes details such as book title, author, publisher, total copies, and available copies. This table helps track book availability.

Members Table

The Members table stores information about library members such as member name, phone number, email, and join date. It helps maintain member records.

Issue Table

The Issue table stores transaction details when a book is issued. It includes book ID, member ID, issue date, due date, and return date.

Fine Table

The Fine table stores fine details for late book returns. It includes fine amount and payment status.

7.2 Table Structure

Books Table

- book_id – INT – Primary Key
- title – VARCHAR(100) – NOT NULL
- author – VARCHAR(100)

- publisher – VARCHAR(100)
- total_copies – INT – NOT NULL
- available_copies – INT – NOT NULL

Members Table

- member_id – INT – Primary Key
- name – VARCHAR(100) – NOT NULL
- phone – VARCHAR(15)
- email – VARCHAR(100)
- join_date – DATE

Issue Table

- issue_id – INT – Primary Key • book_id – INT – Foreign Key
- member_id – INT – Foreign Key
- issue_date – DATE
- due_date – DATE
- return_date – DATE

Fine Table

- fine_id – INT – Primary Key
- issue_id – INT – Foreign Key
- fine_amount – DECIMAL(6,2)
- fine_status – VARCHAR(20)

7.3 Constraints Used

Primary Key

- Ensures each record is unique
- Example: book_id, member_id, issue_id, fine_id

Foreign Key

- Maintains relationship between tables
- Example: Issue.book_id → Books.book_id
- Example: Issue.member_id → Members.member_id

- Example: Fine.issue_id → Issue.issue_id

NOT NULL

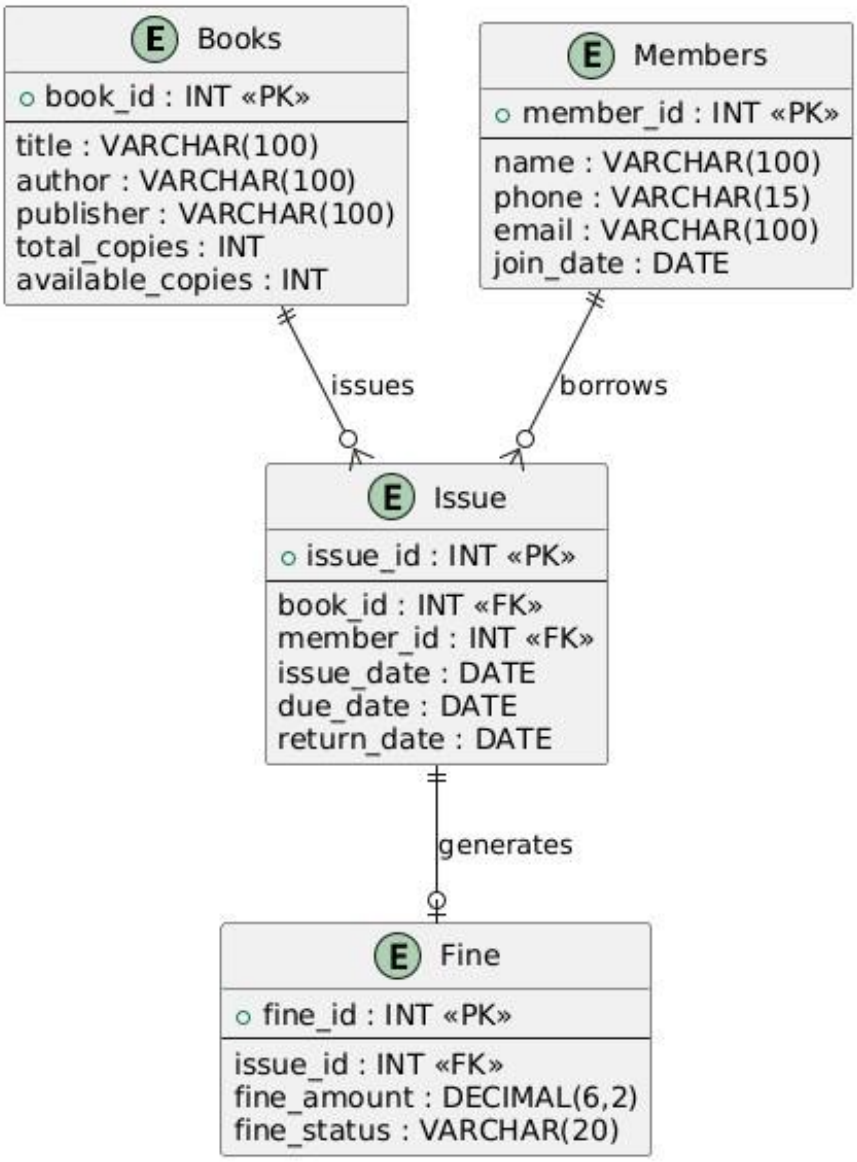
- Ensures important fields cannot be empty
- Example: title, total_copies, available_copies

UNIQUE (Conceptual Use)

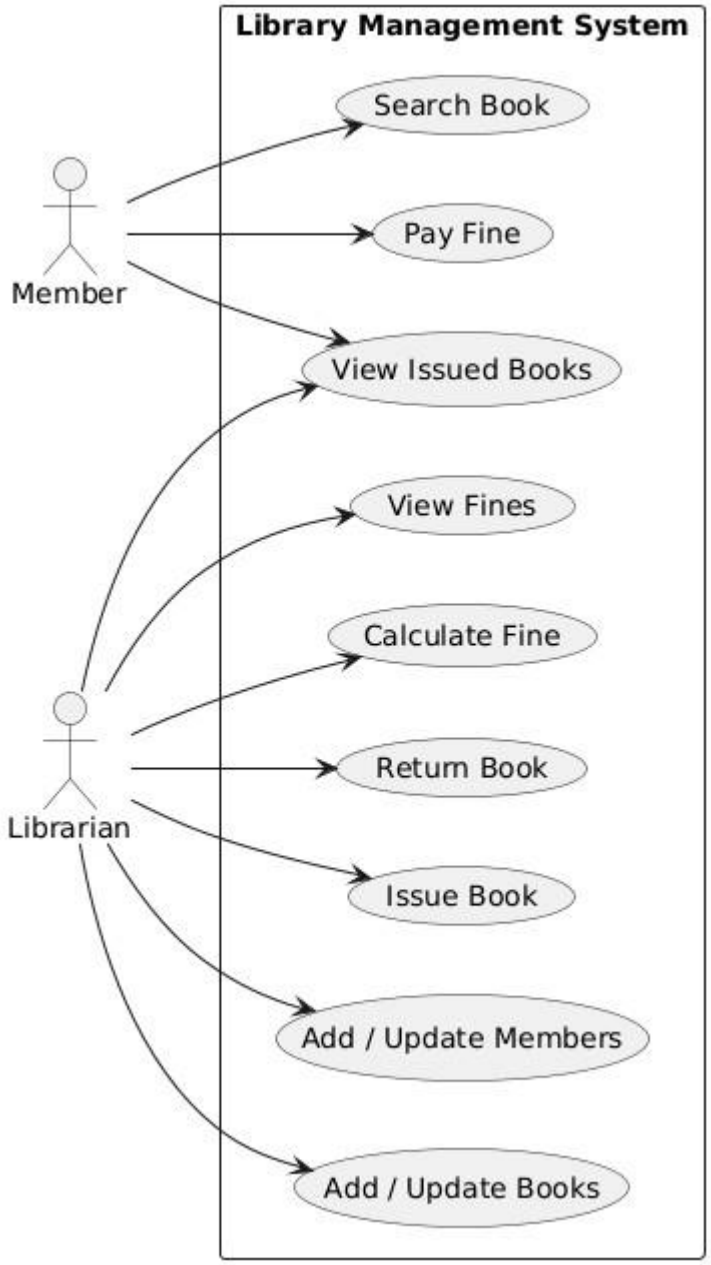
- Prevents duplicate values in specific fields
- Can be applied to email or phone in future system upgrades

8. UML DIAGRAMS

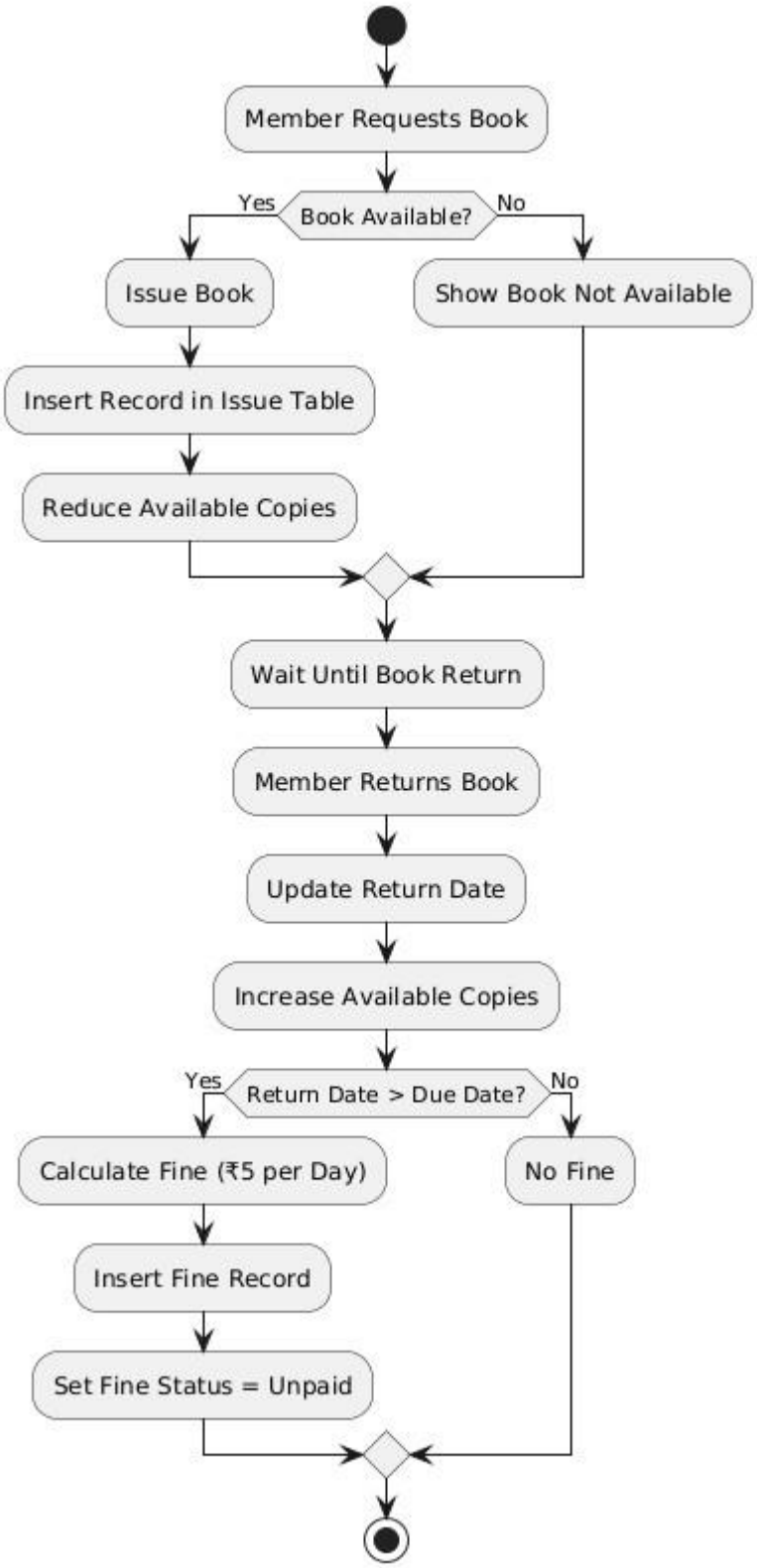
8.1 ER Diagram



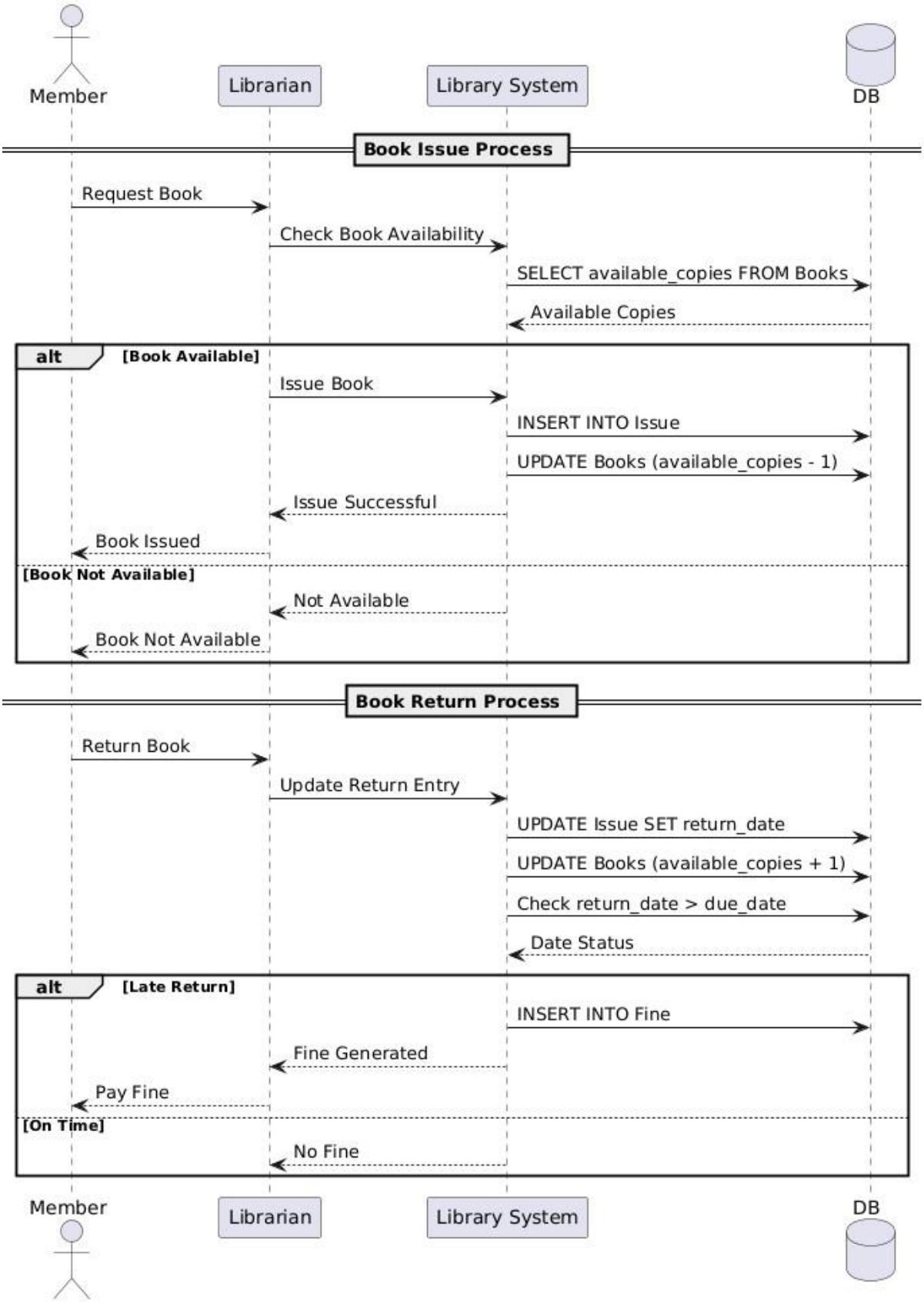
8.2 Use Case Diagram



8.3 Activity Diagram



8.4 Sequence Diagram



CHAPTER 9: SQL IMPLEMENTATION

9.1 Database Creation

```
CREATE DATABASE LibraryDB;
USE LibraryDB;
```

9.2 Table Creation

```
CREATE TABLE Books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(100),
    publisher VARCHAR(100),
    total_copies INT NOT NULL,
    available_copies INT NOT NULL
);

CREATE TABLE Members (
    member_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    email VARCHAR(100),
    join_date DATE
);

CREATE TABLE Issue (
    issue_id INT PRIMARY KEY AUTO_INCREMENT,
    book_id INT,
    member_id INT,
    issue_date DATE,
    due_date DATE,
    return_date DATE,
```

```
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

CREATE TABLE Fine (
    fine_id INT PRIMARY KEY AUTO_INCREMENT,
    issue_id INT,
    fine_amount DECIMAL(6,2),
    fine_status VARCHAR(20),
    FOREIGN KEY (issue_id) REFERENCES Issue(issue_id)
);
```

9.3 Data Insertion

```
INSERT INTO Books(title, author, publisher, total_copies,
available_copies)
VALUES
('Java Programming', 'James Gosling', 'Oracle Press', 10, 10),
('Database Systems', 'Elmasri', 'Pearson', 5, 5),
('Data Structures', 'Seymour Lipschutz', 'McGraw Hill', 8, 8),
('Computer Networks', 'Andrew Tanenbaum', 'Pearson', 6, 6),
('Python Programming', 'Guido van Rossum', 'OReilly', 12, 12),
('Software Engineering', 'Pressman', 'McGraw Hill', 9, 9);

INSERT INTO Members(name, phone, email, join_date)
VALUES
('Abhishek', '9876543210', 'abhi@email.com', CURDATE()),
('Rahul', '9123456780', 'rahul@email.com', CURDATE()),
('Sneha', '9988776655', 'sneha@email.com', CURDATE()),
('Amit', '9012345678', 'amit@email.com', CURDATE()),
('Priya', '9090909090', 'priya@email.com', CURDATE()),
('Rohan', '8888888888', 'rohan@email.com', CURDATE());

START TRANSACTION;

INSERT INTO Issue(book_id, member_id, issue_date, due_date, return_date)
VALUES
(1, 1, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), DATE_ADD(CURDATE(),
INTERVAL 10 DAY)),
(2, 2, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), DATE_ADD(CURDATE(),
```

```
INTERVAL 7 DAY)),
(3, 3, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), NULL),
(4, 4, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), DATE_ADD(CURDATE(),
INTERVAL 9 DAY)),
(5, 5, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), NULL);

UPDATE Books SET available_copies = available_copies - 1 WHERE book_id IN
(1,2,3,4,5);

COMMIT;

START TRANSACTION;

INSERT INTO Fine(issue_id, fine_amount, fine_status)
SELECT issue_id,
DATEDIFF(return_date, due_date) * 5,
'Unpaid'
FROM Issue
WHERE return_date IS NOT NULL
AND return_date > due_date;

COMMIT;
```

9.4 Data Retrieval

```
SELECT Members.name, Books.title, Issue.issue_date, Issue.due_date,
Issue.return_date

FROM Issue

JOIN Members ON Issue.member_id = Members.member_id

JOIN Books ON Issue.book_id = Books.book_id;


SELECT * FROM Fine

WHERE fine_status = 'Unpaid';
```

9.5 Advanced Queries

- GROUP BY

```
SELECT member_id, COUNT(book_id) AS total_books

FROM Issue
```

```
GROUP BY member_id;
```

- HAVING

```
SELECT member_id, COUNT(book_id) AS total_books
FROM Issue
GROUP BY member_id
HAVING COUNT(book_id) > 0;
```

- Subquery

```
SELECT name
FROM Members
WHERE member_id IN (
    SELECT Issue.member_id
    FROM Issue
    JOIN Fine ON Issue.issue_id = Fine.issue_id
    WHERE Fine.fine_status = 'Unpaid'
);
```

- View

Remaining books for each title

```
SELECT title, total_copies, available_copies,
(total_copies - available_copies) AS issued_books
FROM Books;
```

```
CREATE VIEW Issued_Books_View AS
SELECT Members.name, Books.title, Issue.issue_date, Issue.due_date
FROM Issue
JOIN Members ON Issue.member_id = Members.member_id
JOIN Books ON Issue.book_id = Books.book_id;

SELECT * FROM Issued_Books_View;
```

CHAPTER 10: SYSTEM TESTING AND RESULT

```
mysql> SELECT Members.name, Books.title, Issue.issue_date, Issue.due_date, Issue.return_date
    -> FROM Issue
    -> JOIN Members ON Issue.member_id = Members.member_id
    -> JOIN Books ON Issue.book_id = Books.book_id;
+----------+--------------------+------------+------------+-------------+
| name     | title              | issue_date | due_date   | return_date |
+----------+--------------------+------------+------------+-------------+
| Abhishek | Java Programming   | 2026-02-21 | 2026-02-28 | 2026-03-03  |
| Rahul    | Database Systems   | 2026-02-21 | 2026-02-28 | 2026-02-28  |
| Sneha    | Data Structures    | 2026-02-21 | 2026-02-28 | NULL        |
| Amit     | Computer Networks  | 2026-02-21 | 2026-02-28 | 2026-03-02  |
| Priya    | Python Programming | 2026-02-21 | 2026-02-28 | NULL        |
+----------+--------------------+------------+------------+-------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Fine WHERE fine_status = 'Unpaid';
+---------+----------+-------------+-------------+
| fine_id | issue_id | fine_amount | fine_status |
+---------+----------+-------------+-------------+
|       1 |        1 |       15.00 | Unpaid      |
|       2 |        4 |       10.00 | Unpaid      |
+---------+----------+-------------+-------------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT member_id, COUNT(book_id) AS total_books
    -> FROM Issue
    -> GROUP BY member_id;
+-----------+-------------+
| member_id | total_books |
+-----------+-------------+
|         1 |           1 |
|         2 |           1 |
|         3 |           1 |
|         4 |           1 |
|         5 |           1 |
+-----------+-------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT name
    -> FROM Members
    -> WHERE member_id IN (
    ->     SELECT Issue.member_id
    ->     FROM Issue
    ->     JOIN Fine ON Issue.issue_id = Fine.issue_id
    ->     WHERE Fine.fine_status = 'Unpaid'
    -> );
+----------+
| name     |
+----------+
| Abhishek |
| Amit     |
+----------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT title, total_copies, available_copies,
    ->        (total_copies - available_copies) AS issued_books
    -> FROM Books;
+---------------------+--------------+------------------+--------------+
| title               | total_copies | available_copies | issued_books |
+---------------------+--------------+------------------+--------------+
| Java Programming    |           10 |                9 |            1 |
| Database Systems    |            5 |                4 |            1 |
| Data Structures     |            8 |                7 |            1 |
| Computer Networks   |            6 |                5 |            1 |
| Python Programming  |           12 |               11 |            1 |
| Software Engineering |            9 |                9 |            0 |
+---------------------+--------------+------------------+--------------+
6 rows in set (0.00 sec)
```

CHAPTER 11: SECURITY, BACKUP AND RECOVERY

Security, backup, and recovery are essential components of the Library Book Issue & Fine Management System. Since the system stores important information such as book records, member details, issue transactions, and fine data, it is necessary to protect the database from unauthorized access, accidental deletion, and system failures. MySQL provides strong security features and reliable backup and recovery tools that help maintain data safety and system reliability. Implementing proper security ensures that only authorized users can access or modify data. Backup and recovery ensure that data can be restored in case of hardware failure, system crash, or human error. These features help maintain continuous database operation and protect valuable library information.

Security

- **User Privileges**

User privileges define what actions a user can perform on the database. In MySQL, administrators can assign different permissions such as SELECT, INSERT, UPDATE, DELETE, CREATE, and DROP. For example, an admin can have full access to manage database structure and data, while library staff can be given limited access to perform daily operations like inserting issue records or updating return details. This helps prevent unauthorized data modification.

- **Access Control**

Access control ensures that only authorized users can log in and use the database system. MySQL provides authentication using usernames and passwords. Access control also allows restriction of specific tables or database operations. This helps protect sensitive data and ensures data confidentiality and integrity.

Backup

- mysqldump Explanation mysqldump is a MySQL utility tool used to create database backup files. It exports database structure and data into a SQL file. This backup file can be stored safely and used later for recovery. Regular backups help protect data from accidental deletion, hardware failure, or system crash.

Recovery

- **Restore Concept**

Recovery is the process of restoring database data using backup files. If the database is damaged or data is lost, the backup file created using mysqldump can be used to restore the database. Recovery helps maintain system continuity and prevents permanent data loss.

CHAPTER 12: FUTURE SCOPE AND CONCLUSION

The Library Book Issue & Fine Management System provides a strong foundation for managing library records using a relational database. Although the current system focuses mainly on database implementation using MySQL, it can be enhanced further by adding new technologies and advanced features. Future improvements can make the system more user-friendly, automated, and suitable for large-scale organizations. The system currently handles book records, member records, issue transactions, return tracking, and fine calculation efficiently. However, with technological advancements, more intelligent and automated features can be added to improve overall performance and usability. The future scope mainly focuses on expanding the system beyond database-level implementation and integrating it with modern applications and tools.

Future Scope

- Web Integration

The system can be converted into a web-based application using technologies such as PHP, Java, or Python. This will allow users to access the library system through web browsers without directly using MySQL Workbench.

- Mobile Application Integration

A mobile app can be developed so that students and staff can check book availability, due dates, and fine status using smartphones.

- Advanced Reporting System

The system can include graphical reports, charts, and automated report generation for library analysis and management.

- Notification System

Automatic email or SMS notifications can be added to remind members about due dates and fine payments.

- Cloud Database Storage

The database can be hosted on cloud platforms for better accessibility and backup support.

- Data Analytics Features

Advanced analytics can help identify popular books, member reading trends, and library usage statistics.

Conclusion

The Library Book Issue & Fine Management System successfully demonstrates how database technology can be used to manage library operations efficiently. The system reduces manual work, minimizes errors, and improves data accuracy. Using MySQL provides strong data security, fast query processing, and reliable data storage. The project helps in understanding database design, SQL implementation, table relationships, and constraint usage.

Through this project, practical knowledge of database management is gained. It helps in understanding how real-world database systems are designed and implemented. The system ensures efficient data management, faster data retrieval, and reliable record storage.

CHAPTER 13: REFERENCES

- **MySQL Official Documentation – Oracle Corporation**
  Provides complete reference for MySQL commands, database design, transactions, and administration.
- **Elmasri, R., & Navathe, S. B. – Fundamentals of Database Systems**
  Used for understanding database concepts, relational model, normalization, and constraints.
- **Silberschatz, A., Korth, H. F., & Sudarshan, S. – Database System Concepts**
  Reference for database architecture, SQL queries, transactions, and views.
- **MySQL Workbench Official Guide**
  Used for database design, ER diagrams, query execution, and schema management.
- **W3Schools SQL Tutorial**
  Used for learning SQL syntax, SELECT queries, JOINs, GROUP BY, HAVING clauses, and constraints.
- **GeeksforGeeks – Database Management System & SQL Tutorials**
  Used for practical SQL examples, view creation, fine calculation, and DBMS concept explanations.
- **Tutorialspoint – MySQL Tutorial**
  Used for learning MySQL commands, table management, transactions, and database operations.

CHAPTER 14: GLOSSARY

- **DBMS (Database Management System)**
  A software system that allows users to define, create, manage, and maintain databases efficiently. Examples include MySQL, Oracle, and SQL Server.
- **SQL (Structured Query Language)**
  The standard language used to communicate with and manage relational databases. It includes commands like SELECT, INSERT, UPDATE, DELETE, and CREATE.
- **Primary Key**
  A unique identifier for each record in a table. No two rows can have the same primary key value.
- **Foreign Key**
  A field in a table that links to the primary key of another table, ensuring referential integrity between tables.
- **Table**
  A collection of rows and columns in a database that stores related data in a structured format.
- **Record / Row**
  A single entry in a table representing a complete set of related data fields.
- **Column / Field**
  A vertical entity in a table that stores specific data for all records, e.g., book title or member name.
- **View**
  A virtual table created by a SELECT query, allowing users to see specific data from one or more tables without storing it physically.
- **Transaction**
  A sequence of database operations executed as a single unit of work, which must be fully completed or fully rolled back to maintain consistency.
- **Rollback**
  The process of undoing a transaction if an error occurs, restoring the database to its previous consistent state.
- **Commit**
  Finalizing a transaction in the database so that all changes are permanently saved.
- **Normalization**
  The process of organizing data in a database to minimize redundancy and dependency by dividing tables logically.

- **Constraint**
  A rule applied on columns to enforce data integrity. Examples include PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE.
- **JOIN**
  A SQL operation used to combine rows from two or more tables based on a related column between them.
- **ER Diagram (Entity-Relationship Diagram)**
  A visual representation of entities, attributes, and relationships within a database system.
- **HAVING Clause**
  A SQL clause used to filter groups created by GROUP BY, typically based on aggregate functions.
- **Aggregate Functions**
  Functions like COUNT, SUM, AVG, MIN, MAX used to perform calculations on multiple records and return a single value.